Chapter 9. Building Data Pipelines

When people discuss building data pipelines using Apache Kafka, they are usually referring to a couple of use cases. The first is building a data pipeline where Apache Kafka is one of the two end points—for example, getting data from Kafka to S3 or getting data from MongoDB into Kafka. The second use case involves building a pipeline between two different systems but using Kafka as an intermediary. An example of this is getting data from Twitter to Elasticsearch by sending the data first from Twitter to Kafka and then from Kafka to Elasticsearch.

When we added Kafka Connect to Apache Kafka in version 0.9, it was after we saw Kafka used in both use cases at LinkedIn and other large organizations. We noticed that there were specific challenges in integrating Kafka into data pipelines that every organization had to solve, and decided to add APIs to Kafka that solve some of those challenges rather than force every organization to figure them out from scratch.

The main value Kafka provides to data pipelines is its ability to serve as a very large, reliable buffer between various stages in the pipeline. This effectively decouples producers and consumers of data within the pipeline and allows use of the same data from the source in multiple target applications and systems, all with different timeliness and availability requirements. This decoupling, combined with reliability, security, and efficiency, makes Kafka a good fit for most data pipelines.

PUTTING DATA INTEGRATION IN CONTEXT

Some organizations think of Kafka as an *end point* of a pipeline. They look at questions such as "How do I get data from Kafka to Elastic?" This is a valid question to ask—especially if there is data you need in Elastic and it is currently in Kafka—and we will look at ways to do exactly this. But we are going to start the discussion by looking at the use of Kafka within a larger context that includes at least two (and possibly many more) end points that are not Kafka itself. We encourage anyone faced with a data-integration problem to consider the bigger picture and not focus only on the immediate end points. Focusing on short-term integrations is how you end up with a complex and expensive-to-maintain data integration mess.

In this chapter, we'll discuss some of the common issues that you need to take into account when building data pipelines. Those challenges are not specific to Kafka but are general data integration problems. Nonetheless, we will show why Kafka is a good fit for data integration use cases and how it addresses many of those challenges. We will discuss how the Kafka Connect API is different from the normal producer and consumer clients, and when each client type should be used. Then we'll jump into some details of Kafka Connect. While a full discussion of Kafka Connect is outside the scope of this chapter, we will show examples of basic usage to get you started and give you pointers on where to learn more. Finally, we'll discuss other data integration systems and how they integrate with Kafka.

Considerations When Building Data

Pipelines

While we won't get into all the details on building data pipelines here, we would like to highlight some of the most important things to take into account when designing software architectures with the intent of integrating multiple systems.

Timeliness

Some systems expect their data to arrive in large bulks once a day; others expect the data to arrive a few milliseconds after it is generated. Most data pipelines fit somewhere in between these two extremes. Good data integration systems can support different timeliness requirements for different pipelines and also make the migration between different timetables easier as business requirements change. Kafka, being a streaming data platform with scalable and reliable storage, can be used to support anything from near-real-time pipelines to daily batches. Producers can write to Kafka as frequently and infrequently as needed, and consumers can also read and deliver the latest events as they arrive. Or consumers can work in batches: run every hour, connect to Kafka, and read the events that accumulated during the previous hour.

A useful way to look at Kafka in this context is that it acts as a giant buffer that decouples the time-sensitivity requirements between producers and consumers. Producers can write events in real time, while consumers process batches of events, or vice versa. This also makes it trivial to apply back pressure—Kafka itself applies back pressure on producers (by delaying acks when needed) since consumption rate is driven entirely by the consumers.

Reliability

We want to avoid single points of failure and allow for fast and automatic recovery from all sorts of failure events. Data pipelines are often the way data arrives to business-critical systems; failure for more than a few seconds can be hugely disruptive, especially when the timeliness requirement is closer to the few milliseconds end of the spectrum. Another important consideration for reliability is delivery guarantees—some systems can afford to lose data, but most of the time there is a requirement for *at-least-once* delivery, which means every event from the source system will reach its destination, but sometimes retries will cause duplicates. Often, there is even a requirement for *exactly-once* delivery—every event from the source system will reach the destination with no possibility for loss or duplication.

We discussed Kafka's availability and reliability guarantees in depth in Chapter 7. As we discussed, Kafka can provide at-least-once on its own, and exactly-once when combined with an external data store that has a transactional model or unique keys. Since many of the end points are data stores that provide the right semantics for exactly-once delivery, a Kafka-based pipeline can often be implemented as exactly-once. It is worth highlighting that Kafka's Connect API makes it easier for connectors to build an end-to-end exactly-once pipeline by providing an API for integrating with the external systems when handling offsets. Indeed, many of the available open source connectors support exactly-once delivery.

High and Varying Throughput

The data pipelines we are building should be able to scale to very high throughputs, as is often required in modern data systems. Even more importantly, they should be able to adapt if throughput suddenly increases.

With Kafka acting as a buffer between producers and consumers, we no longer need to couple consumer throughput to the producer throughput. We no longer need to implement a complex back-pressure mechanism because if producer throughput exceeds that of the consumer, data will accumulate in Kafka until the consumer can catch up. Kafka's ability to scale by adding consumers or producers independently allows us to scale either side of the pipeline dynamically and independently to match the changing requirements.

Kafka is a high-throughput distributed system—capable of processing hundreds of megabytes per second on even modest clusters—so there is no concern that our pipeline will not scale as demand grows. In addition, the Kafka Connect API focuses on parallelizing the work and can do this on a single node as well as by scaling out, depending on system requirements. We'll describe in the following sections how the platform allows data sources and sinks to split the work among multiple threads of execution and use the available CPU resources even when running on a single machine.

Kafka also supports several types of compression, allowing users and admins to control the use of network and storage resources as the throughput requirements increase.

Data Formats

One of the most important considerations in a data pipeline is reconciling different data formats and data types. The data types supported vary among different databases and other storage systems. You may be loading XMLs and relational data into Kafka, using Avro within Kafka, and then need to convert data to JSON when writing it to Elasticsearch, to Parquet when writing to HDFS, and to CSV when writing to S3.

Kafka itself and the Connect API are completely agnostic when it comes to data formats. As we've seen in previous chapters, producers and consumers can use any serializer to represent data in any format that works for you. Kafka Connect has its own in-memory objects that include data types and schemas, but as we'll soon discuss, it allows for pluggable converters to allow storing these records in any format. This means that no matter which data format you use for Kafka, it does not restrict your choice of connectors.

Many sources and sinks have a schema; we can read the schema from the source with the data, store it, and use it to validate compatibility or even update the schema in the sink database. A classic example is a data pipeline from MySQL to Snowflake. If someone added a column in MySQL, a great pipeline will make sure the column gets added to Snowflake too as we are loading new data into it.

In addition, when writing data from Kafka to external systems, sink connectors are responsible for the format in which the data is written to the external system. Some connectors choose to make this format pluggable. For example, the S3 connector allows a choice between Avro and Parquet formats.

It is not enough to support different types of data. A generic data integration framework should also handle differences in behavior between various sources and sinks. For example, Syslog is a source that pushes data, while relational databases require the framework to pull data out. HDFS is append-only and we can only write data to it, while most systems allow us to both append data and update existing records.

Transformations

Transformations are more controversial than other requirements. There are generally two approaches to building data pipelines: ETL and ELT. ETL, which stands for *Extract-Transform-Load*, means that the data pipeline is responsible for making modifications to the data as it passes through. It has the perceived benefit of saving time and storage because you don't need to store the data, modify it, and store it again. Depending on the transformations, this benefit is sometimes real, but sometimes it shifts the burden of computation and storage to the data pipeline itself, which may or may not be desirable. The main drawback of this approach is that the transformations that happen to the data in the pipeline may tie the hands of those who wish to process the data further down the pipe. If the person who built the pipeline between MongoDB and MySQL decided to filter certain events or remove fields from records, all the users and applications who access the data in MySQL will only have access to partial data. If they require access to the missing fields, the pipeline needs to be rebuilt, and historical data will require reprocessing (assuming it is available).

ELT stands for *Extract-Load-Transform* and means that the data pipeline does only minimal transformation (mostly around data type conversion), with the goal of making sure the data that arrives at the target is as similar as possible to the source data. In these systems, the target system collects "raw data" and all required processing is done at the target system. The benefit here is that the system provides maximum flexibility to users of the target system, since they have access to all the data. These systems also tend to be easier to troubleshoot since all data processing is limited to one system rather than split between the pipeline and additional applications. The drawback is that the transformations take CPU and storage resources at the target system. In some cases, these systems are expensive and there is strong motivation to move computation off those systems when possible.

Kafka Connect includes the Single Message Transformation feature, which transforms records while they are being copied from a source to Kafka, or from Kafka to a target. This includes routing messages to different topics, filtering messages, changing data types, redacting specific fields, and more. More complex transformations that involve joins and aggregations are typically done using Kafka Streams, and we will explore those in detail in a separate chapter.

WARNING

When building an ETL system with Kafka, keep in mind that Kafka allows you to build one-to-many pipelines, where the source data is written to Kafka once and then consumed by multiple applications and written to multiple target systems. Some preprocessing and cleanup is expected, such as standardizing timestamps and data types, adding lineage, and perhaps removing personal information—transformations that will benefit all consumers of the data. But don't prematurely clean and optimize the data on ingest because it might be needed less refined elsewhere.

Security

Security should always be a concern. In terms of data pipelines, the main security concerns are usually:

- Who has access to the data that is ingested into Kafka?
- Can we make sure the data going through the pipe is encrypted? This is mainly a concern for data pipelines that cross datacenter boundaries.
- Who is allowed to make modifications to the pipelines?
- If the data pipeline needs to read or write from access-controlled locations, can it authenticate properly?
- Is our PII (Personally Identifiable Information) handling compliant with laws and regulations regarding its storage, access and use?

Kafka allows encrypting data on the wire, as it is piped from sources to Kafka and from Kafka to sinks. It also supports authentication (via SASL) and authorization—so you can be sure that if a topic contains sensitive information, it can't be piped into less secured systems by someone unauthorized. Kafka also provides an audit log to track access—unauthorized and authorized. With some extra coding, it is also possible to track where the events in each topic came from and who modified them, so you can provide the entire lineage for each record.

Kafka security is discussed in detail in Chapter 11. However, Kafka Connect and its connectors need to be able to connect to, and authenticate with, external data systems, and configuration of connectors will include credentials for authenticating with external data systems.

These days it is not recommended to store credentials in configuration files, since this means that the configuration files have to be handled with extra care and have restricted access. A common solution is to use an external secret management system such as HashiCorp Vault. Kafka Connect includes support for external secret configuration. Apache Kafka only includes the framework that allows introduction of pluggable external config providers, an example provider that reads configuration from a file, and there are community-developed external config providers that integrate with Vault, AWS, and Azure.

Failure Handling

Assuming that all data will be perfect all the time is dangerous. It is important to plan for failure handling in advance. Can we prevent faulty records from ever making it into the pipeline? Can we recover from records that cannot be parsed? Can bad records get fixed (perhaps by a human) and reprocessed? What if the bad event looks exactly like a normal event and you only discover the problem a few days later?

Because Kafka can be configured to store all events for long periods of time, it is possible to go back in time and recover from errors when needed. This also allows replaying the events stored in Kafka to the target system if they were lost.

Coupling and Agility

A desirable characteristic of data pipeline implementation is to decouple the data sources and data targets. There are multiple ways accidental coupling can happen:

Ad hoc pipelines

Some companies end up building a custom pipeline for each pair of applications they want to connect. For example, they use Logstash to dump logs to Elasticsearch, Flume to dump logs to HDFS, Oracle GoldenGate to get data from Oracle to HDFS, Informatica to get data from MySQL and XML to Oracle, and so on. This tightly couples the data pipeline to the specific end points and creates a mess of integration points that requires significant effort to deploy, maintain, and monitor. It also means that every new system the company adopts will require building additional pipelines, increasing the cost of adopting new technology, and inhibiting innovation.

Loss of metadata

If the data pipeline doesn't preserve schema metadata and does not allow for schema evolution, you end up tightly coupling the software producing the data at the source and the software that uses it at the destination. Without schema information, both software products need to include information on how to parse the data and interpret it. If data flows from Oracle to HDFS and a DBA added a new field in Oracle without preserving schema information and allowing schema evolution, either every app that reads data from HDFS will break or all the developers will need to upgrade their applications at the same time. Neither option is agile. With support for schema evolution in the pipeline, each team can modify their applications at their own pace without worrying that things will break down the line.

Extreme processing

As we mentioned when discussing data transformations, some processing of data is inherent to data pipelines. After all, we are moving data between different systems where different data formats make sense and different use cases are supported. However, too much processing ties all the downstream systems to decisions made when building the pipelines about which fields to preserve, how to aggregate data, etc. This often leads to constant changes to the pipeline as requirements of downstream applications change, which isn't agile, efficient, or safe. The more agile way is to preserve as much of the raw data as possible and allow downstream apps, including Kafka Streams apps, to make their own decisions regarding data processing and aggregation.

When to Use Kafka Connect Versus Producer and Consumer

When writing to Kafka or reading from Kafka, you have the choice between using traditional producer and consumer clients, as described in Chapters 3 and 4, or using the Kafka Connect API and the connectors, as we'll describe in the following sections. Before we start diving into the details of Kafka Connect, you may already be wondering, "When do I use which?"

As we've seen, Kafka clients are clients embedded in your own application. It allows your application to write data to Kafka or to read data from Kafka. Use Kafka clients when you can modify the code of the application that you want to connect an application to and when you want to either push data into Kafka or pull data from Kafka.

You will use Connect to connect Kafka to datastores that you did not write and whose code or APIs you cannot or will not modify. Connect will be used to pull data from the external datastore into Kafka or push data from Kafka to an external store. To use Kafka Connect, you need a connector for the datastore to which you want to connect, and nowadays these connectors are plentiful. This means that in practice, users of Kafka Connect only need to write configuration files.

If you need to connect Kafka to a datastore and a connector does not exist yet, you can choose between writing an app using the Kafka clients or the Connect API. Connect is recommended because it provides out-of-the-box features like configuration management, offset storage, parallelization, error handling, support for different data types, and standard management REST APIs. Writing a small app that connects Kafka to a datastore sounds simple, but there are many little details you will need to handle concerning data types and configuration that make the task nontrivial. What's more, you will need to maintain this pipeline app and document it, and your teammates will need to learn how to use it. Kafka Connect is a standard part of the Kafka ecosystem, and it handles most of this for you, allowing you to focus on transporting data to and from the external stores.

Kafka Connect

Kafka Connect is a part of Apache Kafka and provides a scalable and reliable way to copy data between Kafka and other datastores. It provides APIs and a runtime to develop and run *connector plug-ins*—libraries that Kafka Connect executes and that are responsible for moving the data. Kafka Connect runs as a cluster of *worker processes*. You install the connector plug-ins on the workers and then use a REST API to configure and manage *connectors*, which run with a specific configuration. *Connectors* start additional *tasks* to move large amounts of data in parallel and use the available resources on the worker nodes more efficiently. Source connector tasks just need to read data from the source system and provide Connect data objects to the worker processes. Sink connector tasks get connector data objects from the workers and are responsible for writing them to the target data system. Kafka Connect uses *convertors* to support storing those data objects in Kafka in different formats—JSON format support is part of Apache Kafka, and the Confluent Schema Registry provides Avro, Protobuf, and JSON Schema converters. This allows users to choose the format in which data is stored in Kafka independent of the connectors they use, as well as how the schema of the data is handled (if at all).

This chapter cannot possibly get into all the details of Kafka Connect and its many connectors. This could fill an entire book on its own. We will, however, give an overview of Kafka Connect and how to use it, and point to additional resources for reference.

Running Kafka Connect

Kafka Connect ships with Apache Kafka, so there is no need to install it separately. For production use, especially if you are planning to use Connect to move large amounts of data or run many connectors, you should run Connect on separate servers from your Kafka brokers. In this case, install Apache Kafka on all the machines, and simply start the brokers on some servers and start Connect on other servers.

Starting a Connect worker is very similar to starting a broker—you call the start script with a properties file:

```
bin/connect-distributed.sh config/connect-distributed.properties
```

There are a few key configurations for Connect workers:

`bootstrap.servers`

A list of Kafka brokers that Connect will work with. `Connectors` will pipe their data either to or from those brokers. You don't need to specify every broker in the cluster, but it's recommended to specify at least three.

`group.id`

All workers with the same group ID are part of the same Connect cluster. A connector started on the cluster will run on any worker, and so will its tasks.

`plugin.path`

Kafka Connect uses a pluggable architecture where connectors, converters, transformations, and secret providers can be downloaded and added to the platform. In order to do this, Kafka Connect has to be able to find and load those plug-ins.

We can configure one or more directories as locations where connectors and their dependencies can be found. For example, we can configure `plugin.path=/opt/connectors,/home/gwenshap/connectors`. Inside one of these directories, we will typically create a subdirectory for each connector, so in the previous example, we'll create `/opt/connectors/jdbc` and `/opt/connectors/elastic`. Inside each subdirectory, we'll place the connector jar itself and all its dependencies. If the connector ships as an `uberJar` and has no dependencies, it can be placed directly in `plugin.path` and doesn't require a subdirectory. But note that placing dependencies in the top-level path will not work.

An alternative is to add the connectors and all their dependencies to the Kafka Connect classpath, but this is not recommended and can introduce errors if you use a connector that brings a dependency that conflicts with one of Kafka's dependencies. The recommended approach is to use `plugin.path` configuration.

`key.converter` and `value.converter`

Connect can handle multiple data formats stored in Kafka. The two configurations set the converter for the key and value part of the message that will be stored in Kafka. The default is JSON format using the `JSONConverter` included in Apache Kafka. These configurations can also be set to `AvroConverter`, `ProtobufConverter`, or `JscoSchemaConverter`, which are part of the Confluent Schema Registry.

Some converters include converter-specific configuration parameters. You need to prefix these parameters with `key.converter.` or `value.converter.`, depending on whether you want to apply them to the key or value converter. For example, JSON messages can include a schema or be schema-less. To support either, you can set `key.converter.schemas.enable=true` or `false`, respec-

tively. The same configuration can be used for the value converter by setting `value.converter.schemas.enable` to `true` or `false`. Avro messages also contain a schema, but you need to configure the location of the Schema Registry using `key.converter.schema.registry.url` and `value.converter.schema.registry.url`.

`rest.host.name` and `rest.port`

Connectors are typically configured and monitored through the REST API of Kafka Connect. You can configure the specific port for the REST API.

Once the workers are up and you have a cluster, make sure it is up and running by checking the REST API:

```
$ curl http://localhost:8083/
{"version":"3.0.0-SNAPSHOT","commit":"fae0784ce32a448a","kafka_cluster_id":"pfkYIGZQSXm8
```

Accessing the base REST URI should return the current version you are running. We are running a snapshot of Kafka 3.0.0 (prerelease). We can also check which connector plug-ins are available:

```
$ curl http://localhost:8083/connector-plugins

[
  {
    "class": "org.apache.kafka.connect.file.FileStreamSinkConnector",
    "type": "sink",
    "version": "3.0.0-SNAPSHOT"
  },
  {
    "class": "org.apache.kafka.connect.file.FileStreamSourceConnector",
    "type": "source",
    "version": "3.0.0-SNAPSHOT"
  },
  {
    "class": "org.apache.kafka.connect.mirror.MirrorCheckpointConnector",
    "type": "source",
    "version": "1"
  },
  {
    "class": "org.apache.kafka.connect.mirror.MirrorHeartbeatConnector",
    "type": "source",
    "version": "1"
  },
  {
    "class": "org.apache.kafka.connect.mirror.MirrorSourceConnector",
    "type": "source",
    "version": "1"
  }
]
```

We are running plain Apache Kafka, so the only available connector plug-ins are the file source, file sink, and the connectors that are part of MirrorMaker 2.0.

Let's see how to configure and use these example connectors, and then we'll dive into more advanced examples that require setting up external data systems to connect to.

STANDALONE MODE

Take note that Kafka Connect also has a standalone mode. It is similar to distributed mode—you just run `bin/connect-standalone.sh` instead of `bin/connect-distributed.sh`. You can also pass in a connector configuration file on the command line instead of through the REST API. In this mode, all the connectors and tasks run on the one standalone worker. It is used in cases where connectors and tasks need to run on a specific machine (e.g., the `syslog` connector listens on a port, so you need to know which machines it is running on).

Connector Example: File Source and File Sink

This example will use the file connectors and JSON converter that are part of Apache Kafka. To follow along, make sure you have ZooKeeper and Kafka up and running.

To start, let's run a distributed Connect worker. In a real production environment, you'll want at least two or three of these running to provide high availability. In this example, we'll only start one:

```
bin/connect-distributed.sh config/connect-distributed.properties &
```

Now it's time to start a file source. As an example, we will configure it to read the Kafka configuration file—basically piping Kafka's configuration into a Kafka topic:

```
echo '{"name":"load-kafka-config", "config":{"connector.class":
"FileStreamSource","file":"config/server.properties","topic":
"kafka-config-topic"}}' | curl -X POST -d @- http://localhost:8083/connectors
-H "Content-Type: application/json"

{
  "name": "load-kafka-config",
  "config": {
    "connector.class": "FileStreamSource",
    "file": "config/server.properties",
    "topic": "kafka-config-topic",
    "name": "load-kafka-config"
  },
  "tasks": [
    {
      "connector": "load-kafka-config",
      "task": 0
    }
  ],
  "type": "source"
}
```

To create a connector, we wrote a JSON that includes a connector name, `load-kafka-config`, and a connector configuration map, which includes the connector class, the file we want to load, and the topic we want to load the file into.

Let's use the Kafka Console consumer to check that we have loaded the configuration into a topic:

```
gwen$ bin/kafka-console-consumer.sh --bootstrap-server=localhost:9092
--topic kafka-config-topic --from-beginning
```

If all went well, you should see something along the lines of:

```
{"schema":{"type":"string","optional":false},"payload":"# Licensed to the Apache Softwar

<more stuff here>

{"schema":{"type":"string","optional":false},"payload":"############################ Se
{"schema":{"type":"string","optional":false},"payload":""}
{"schema":{"type":"string","optional":false},"payload":"# The id of the broker. This mus
{"schema":{"type":"string","optional":false},"payload":"broker.id=0"}
{"schema":{"type":"string","optional":false},"payload":""}

<more stuff here>
```

This is literally the contents of the *config/server.properties* file, as it was converted to JSON line by line and placed in `kafka-config-topic` by our connector. Note that by default, the JSON converter places a schema in each record. In this specific case, the schema is very simple—there is only a single column, named `payload` of type `string`, and it contains a single line from the file for each record.

Now let's use the file sink converter to dump the contents of that topic into a file. The resulting file should be completely identical to the original *server.properties* file, as the JSON converter will convert the JSON records back into simple text lines:

```
echo '{"name":"dump-kafka-config", "config":{"connector.class":"FileStreamSink","file":"

{"name":"dump-kafka-config","config":{"connector.class":"FileStreamSink","file":"copy-of
```

Note the changes from the source configuration: the class we are using is now `FileStreamSink` rather than `FileStreamSource`. We still have a file property, but now it refers to the destination file rather than the source of the records, and instead of specifying a *topic*, you specify *topics*. Note the plurality—you can write multiple topics into one file with the sink, while the source only allows writing into one topic.

If all went well, you should have a file named *copy-of-server-properties*, which is completely identical to the *config/server.properties* we used to populate `kafka-config-topic`.

To delete a connector, you can run:

```
curl -X DELETE http://localhost:8083/connectors/dump-kafka-config
```

WARNING

This example uses FileStream connectors because they are simple and built into Kafka, allowing you to create your first pipeline without installing anything except Kafka. These should not be used for actual production pipelines, as they have many limitations and no reliability guarantees. There are several alternatives you can use if you want to ingest data from files: FilePulse Connector, FileSystem Connector, or SpoolDir.

Connector Example: MySQL to Elasticsearch

Now that we have a simple example working, let's do something more useful. Let's take a MySQL table, stream it to a Kafka topic, and from there load it to Elasticsearch and index its content.

We are running tests on a MacBook. To install MySQL and Elasticsearch, simply run:

```
brew install mysql
brew install elasticsearch
```

The next step is to make sure you have the connectors. There are a few options:

1. Download and install using Confluent Hub client.
2. Download from the Confluent Hub website (or from any other website where the connector you are interested in is hosted).
3. Build from source code. To do this, you'll need to:
   1. Clone the connector source:

      ```
      git clone https://github.com/confluentinc/kafka-connect-elasticsearch
      ```

   2. Run `mvn install -DskipTests` to build the project.
   3. Repeat with the JDBC connector.

Now we need to load these connectors. Create a directory, such as */opt/connectors* and update *config/connect-distributed.properties* to include `plugin.path=/opt/connectors`.

Then take the jars that were created under the `target` directory where you built each connector and copy each one, plus their dependencies, to the appropriate subdirectories of `plugin.path`:

```
gwen$ mkdir /opt/connectors/jdbc
gwen$ mkdir /opt/connectors/elastic
gwen$ cp .../kafka-connect-jdbc/target/kafka-connect-jdbc-10.3.x-SNAPSHOT.jar /opt/conne
gwen$ cp ../kafka-connect-elasticsearch/target/kafka-connect-elasticsearch-11.1.0-SNAPSH
gwen$ cp ../kafka-connect-elasticsearch/target/kafka-connect-elasticsearch-11.1.0-SNAPSH
```

In addition, since we need to connect not just to any database but specifically to MySQL, you'll need to download and install a MySQL JDBC driver. The driver doesn't ship with the connector for license reasons. You can download the driver from the MySQL website and then place the jar in */opt/connectors/jdbc*.

Restart the Kafka Connect workers and check that the new connector plug-ins are listed:

```
gwen$  bin/connect-distributed.sh config/connect-distributed.properties &

gwen$  curl http://localhost:8083/connector-plugins
[
  {
    "class": "io.confluent.connect.elasticsearch.ElasticsearchSinkConnector",
    "type": "sink",
    "version": "11.1.0-SNAPSHOT"
  },
  {
    "class": "io.confluent.connect.jdbc.JdbcSinkConnector",
    "type": "sink",
    "version": "10.3.x-SNAPSHOT"
  },
  {
    "class": "io.confluent.connect.jdbc.JdbcSourceConnector",
    "type": "source",
```

```
    "version": "10.3.x-SNAPSHOT"
  }
```

We can see that we now have additional connector plug-ins available in our `Connect` cluster.

The next step is to create a table in MySQL that we can stream into Kafka using our JDBC connector:

```
gwen$ mysql.server restart
gwen$  mysql --user=root

mysql> create database test;
Query OK, 1 row affected (0.00 sec)

mysql> use test;
Database changed
mysql> create table login (username varchar(30), login_time datetime);
Query OK, 0 rows affected (0.02 sec)

mysql> insert into login values ('gwenshap', now());
Query OK, 1 row affected (0.01 sec)

mysql> insert into login values ('tpalino', now());
Query OK, 1 row affected (0.00 sec)
```

As you can see, we created a database and a table, and inserted a few rows as an example.

The next step is to configure our JDBC source connector. We can find out which configuration options are available by looking at the documentation, but we can also use the REST API to find the available configuration options:

```
gwen$ curl -X PUT -d '{"connector.class":"JdbcSource"}' localhost:8083/connector-plugins

{
    "configs": [
        {
            "definition": {
                "default_value": "",
                "dependents": [],
                "display_name": "Timestamp Column Name",
                "documentation": "The name of the timestamp column to use
                to detect new or modified rows. This column may not be
                nullable.",
                "group": "Mode",
                "importance": "MEDIUM",
                "name": "timestamp.column.name",
                "order": 3,
                "required": false,
                "type": "STRING",
                "width": "MEDIUM"
            },
            <more stuff>
```

We asked the REST API to validate configuration for a connector and sent it a configuration with just the class name (this is the bare minimum configuration necessary). As a response, we got the JSON definition of all available configurations.

With this information in mind, it's time to create and configure our JDBC connector:

```
echo '{"name":"mysql-login-connector", "config":{"connector.class":"JdbcSourceConnector"
```

```
{
  "name": "mysql-login-connector",
  "config": {
    "connector.class": "JdbcSourceConnector",
    "connection.url": "jdbc:mysql://127.0.0.1:3306/test?user=root",
    "mode": "timestamp",
    "table.whitelist": "login",
    "validate.non.null": "false",
    "timestamp.column.name": "login_time",
    "topic.prefix": "mysql.",
    "name": "mysql-login-connector"
  },
  "tasks": []
}
```

Let's make sure it worked by reading data from the `mysql.login` topic:

```
gwen$ bin/kafka-console-consumer.sh --bootstrap-server=localhost:9092 --topic mysql.logi
```

If you get errors saying the topic doesn't exist or you see no data, check the Connect worker logs for errors such as:

```
[2016-10-16 19:39:40,482] ERROR Error while starting connector mysql-login-connector (or
org.apache.kafka.connect.errors.ConnectException: java.sql.SQLException: Access denied f
        at io.confluent.connect.jdbc.JdbcSourceConnector.start(JdbcSourceConnector.java:
```

Other issues can involve the existence of the driver in the classpath or permissions to read the table.

Once the connector is running, if you insert additional rows in the `login` table, you should immediately see them reflected in the `mysql.login` topic.

CHANGE DATA CAPTURE AND DEBEZIUM PROJECT

The JDBC connector that we are using uses JDBC and SQL to scan database tables for new records. It detects new records by using timestamp fields or an incrementing primary key. This is a relatively inefficient and at times inaccurate process. All relational databases have a transaction log (also called redo log, binlog, or write-ahead log) as part of their implementation, and many allow external systems to read data directly from their transaction log—a far more accurate and efficient process known as `change data capture`. Most modern ETL systems depend on change data capture as a data source. The Debezium Project provides a collection of high-quality, open source, change capture connectors for a variety of databases. If you are planning on streaming data from a relational database to Kafka, we highly recommend using a Debezium change capture connector if one exists for your database. In addition, the Debezium documentation is one of the best we've seen—in addition to documenting the connectors themselves, it covers useful design patterns and use cases related to change data capture, especially in the context of microservices.

Getting MySQL data to Kafka is useful in itself, but let's make things more fun by writing the data to Elasticsearch.

First, we start Elasticsearch and verify it is up by accessing its local port:

```
gwen$ elasticsearch &
gwen$ curl http://localhost:9200/
{
  "name" : "Chens-MBP",
  "cluster_name" : "elasticsearch_gwenshap",
  "cluster_uuid" : "X69zu3_sQNGb7zbMh7NDVw",
  "version" : {
    "number" : "7.5.2",
    "build_flavor" : "default",
    "build_type" : "tar",
    "build_hash" : "8bec50e1e0ad29dad5653712cf3bb580cd1afcdf",
    "build_date" : "2020-01-15T12:11:52.313576Z",
    "build_snapshot" : false,
    "lucene_version" : "8.3.0",
    "minimum_wire_compatibility_version" : "6.8.0",
    "minimum_index_compatibility_version" : "6.0.0-beta1"
  },
  "tagline" : "You Know, for Search"
}
```

Now create and start the connector:

```
echo '{"name":"elastic-login-connector", "config":{"connector.class":"ElasticsearchSinkC

{
  "name": "elastic-login-connector",
  "config": {
    "connector.class": "ElasticsearchSinkConnector",
    "connection.url": "http://localhost:9200",
    "topics": "mysql.login",
    "key.ignore": "true",
    "name": "elastic-login-connector"
  },
  "tasks": [
    {
      "connector": "elastic-login-connector",
      "task": 0
    }
  ]
}
```

There are a few configurations we need to explain here. The `connection.url` is simply the URL of the local Elasticsearch server we configured earlier. Each topic in Kafka will become, by default, a separate Elasticsearch index, with the same name as the topic. The only topic we are writing to Elasticsearch is `mysql.login`. The JDBC connector does not populate the message key. As a result, the events in Kafka have null keys. Because the events in Kafka lack keys, we need to tell the Elasticsearch connector to use the topic name, partition ID, and offset as the key for each event. This is done by setting `key.ignore` configuration to `true`.

Let's check that the index with `mysql.login` data was created:

```
gwen$ curl 'localhost:9200/_cat/indices?v'
health status index       uuid                   pri rep docs.count docs.deleted store.s
yellow open   mysql.login wkeyk9-bQea6NJmAFjv4hw   1   1          2            0      3.
```

If the index isn't there, look for errors in the Connect worker log. Missing configurations or libraries are common causes for errors. If all is well, we can search the index for our records:

```
gwen$ curl -s -X "GET" "http://localhost:9200/mysql.login/_search?pretty=true"
{
  "took" : 40,
  "timed_out" : false,
  "_shards" : {
    "total" : 1,
    "successful" : 1,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : {
      "value" : 2,
      "relation" : "eq"
    },
    "max_score" : 1.0,
    "hits" : [
      {
        "_index" : "mysql.login",
        "_type" : "_doc",
        "_id" : "mysql.login+0+0",
        "_score" : 1.0,
        "_source" : {
          "username" : "gwenshap",
          "login_time" : 1621699811000
        }
      },
      {
        "_index" : "mysql.login",
        "_type" : "_doc",
        "_id" : "mysql.login+0+1",
        "_score" : 1.0,
        "_source" : {
          "username" : "tpalino",
          "login_time" : 1621699816000
        }
      }
    ]
  }
}
```

If you add new records to the table in MySQL, they will automatically appear in the `mysql.login` topic in Kafka and in the corresponding Elasticsearch index.

Now that we've seen how to build and install the JDBC source and Elasticsearch sink, we can build and use any pair of connectors that suits our use case. Confluent maintains a set of their own prebuilt connectors, as well as some from across the community and other vendors, at Confluent Hub. You can pick any connector on the list that you wish to try out, download it, configure it—either based on the documentation or by pulling the configuration from the REST API—and run it on your Connect worker cluster.

BUILD YOUR OWN CONNECTORS

The Connector API is public and anyone can create a new connector. So if the datastore you wish to integrate with does not have an existing connector, we encourage you to write your own. You can then contribute it to Confluent Hub so others can discover and use it. It is beyond the scope of this chapter to discuss all the details involved in building a connector, but there are multiple blog posts that explain how to do so, and good talks from Kafka Summit NY 2019, Kafka Summit London 2018, and ApacheCon. We also recommend looking at the existing connectors as a starting point and perhaps jump-starting using an Apache Maven archtype. We always encourage you to ask for assistance or show off your latest connectors on the Apache Kafka community mailing list (users@kafka.apache.org) or submit them to Confluent Hub so they can be easily found.

Single Message Transformations

Copying records from MySQL to Kafka and from there to Elastic is rather useful on its own, but ETL pipelines typically involve a transformation step. In the Kafka ecosystem we separate transformations to single message transformations (SMTs), which are stateless, and stream processing, which can be stateful. SMTs can be done within Kafka Connect transforming messages while they are being copied, often without writing any code. More complex transformations, which typically involve joins or aggregation, will require the stateful Kafka Streams framework. We'll discuss Kafka Streams in a later chapter.

Apache Kafka includes the following SMTs:

Cast

Change data type of a field.

MaskField

Replace the contents of a field with null. This is useful for removing sensitive or personally identifying data.

Filter

Drop or include all messages that match a specific condition. Built-in conditions include matching on a topic name, a particular header, or whether the message is a tombstone (that is, has a null value).

Flatten

Transform a nested data structure to a flat one. This is done by concatenating all the names of all fields in the path to a specific value.

HeaderFrom

Move or copy fields from the message into the header.

InsertHeader

Add a static string to the header of each message.

InsertField

Add a new field to a message, either using values from its metadata such as offset, or with a static value.

RegexRouter

Change the destination topic using a regular expression and a replacement string.

ReplaceField

Remove or rename a field in the message.

TimestampConverter

Modify the time format of a field—for example, from Unix Epoch to a String.

TimestampRouter

Modify the topic based on the message timestamp. This is mostly useful in sink connectors when we want to copy messages to specific table partitions based on their timestamp and the topic field is used to find an equivalent dataset in the destination system.

In addition, transformations are available from contributors outside the main Apache Kafka code base. Those can be found on GitHub (Lenses.io, Aiven, and Jeremy Custenborder have useful collections) or on Confluent Hub.

To learn more about Kafka Connect SMTs, you can read detailed examples of many transformations in the "Twelve Days of SMT" blog series. In addition, you can learn how to write your own transformations by following a tutorial and deep dive.

As an example, let's say that we want to add a record header to each record produced by the MySQL connector we created previously. The header will indicate that the record was created by this MySQL connector, which is useful in case auditors want to examine the lineage of these records.

To do this, we'll replace the previous MySQL connector configuration with the following:

```
echo '{
  "name": "mysql-login-connector",
  "config": {
    "connector.class": "JdbcSourceConnector",
    "connection.url": "jdbc:mysql://127.0.0.1:3306/test?user=root",
    "mode": "timestamp",
    "table.whitelist": "login",
    "validate.non.null": "false",
    "timestamp.column.name": "login_time",
    "topic.prefix": "mysql.",
    "name": "mysql-login-connector",
    "transforms": "InsertHeader",
    "transforms.InsertHeader.type":
      "org.apache.kafka.connect.transforms.InsertHeader",
    "transforms.InsertHeader.header": "MessageSource",
    "transforms.InsertHeader.value.literal": "mysql-login-connector"
  }}' | curl -X POST -d @- http://localhost:8083/connectors --header "content-Type:appli
```

Now, if you insert a few more records into the MySQL table that we created in the previous example, you'll be able to see that the new messages in the `mysql.login` topic have headers (note that you'll need Apache Kafka 2.7 or higher to print headers in the console consumer):

```
bin/kafka-console-consumer.sh --bootstrap-server=localhost:9092 --topic mysql.login --fr

NO_HEADERS      {"schema":{"type":"struct","fields":[{"type":"string","optional":true,"f
MessageSource:mysql-login-connector     {"schema":{"type":"struct","fields":

[{"type":"string","optional":true,"field":"username"},{"type":"int64","optional":true,"n
```

As you can see, the old records show `NO_HEADERS`, but the new records show `MessageSource:mysql-login-connector`.

ERROR HANDLING AND DEAD LETTER QUEUES

Transforms is an example of a connector config that isn't specific to one connector but can be used in the configuration of any connector. Another very useful connector configuration that can be used in any sink connector is `error.tolerance`—you can configure any connector to silently drop corrupt messages, or to route them to a special topic called a "dead letter queue." You can find more details in the "Kafka Connect Deep Dive—Error Handling and Dead Letter Queues" blog post.

A Deeper Look at Kafka Connect

To understand how Kafka Connect works, you need to understand three basic concepts and how they interact. As we explained earlier and demonstrated with examples, to use Kafka Connect, you need to run a cluster of workers and create/remove connectors. An additional detail we did not dive into before is the handling of data by converters—these are the components that convert MySQL rows to JSON records, which the connector wrote into Kafka.

Let's look a bit deeper into each system and how they interact with one another.

Connectors and tasks

Connector plug-ins implement the Connector API, which includes two parts:

Connectors

The connector is responsible for three important things:

- Determining how many tasks will run for the connector
- Deciding how to split the data-copying work between the tasks
- Getting configurations for the tasks from the workers and passing them along

For example, the JDBC source connector will connect to the database, discover the existing tables to copy, and based on that decide how many tasks are needed—choosing the lower of `tasks.max` configuration and the number of tables. Once it decides how many tasks will run, it will generate a configuration for each task—using both the connector configuration (e.g., `connection.url`) and a list of tables it assigns for each task to copy. The `taskConfigs()` method returns a list of maps (i.e., a configuration for each task we want to run). The workers are then responsible for starting the tasks and giving each one its own unique configuration so that it will copy a unique subset of tables from the database. Note that when you start the connector via the REST API, it may start on any node, and subsequently the tasks it starts may also execute on any node.

Tasks

Tasks are responsible for actually getting the data in and out of Kafka. All tasks are initialized by receiving a context from the worker. Source context includes an object that allows the source task to store the offsets of source records (e.g., in the file connector,

the offsets are positions in the file; in the JDBC source connector, the offsets can be a timestamp column in a table). Context for the sink connector includes methods that allow the connector to control the records it receives from Kafka—this is used for things like applying back pressure and retrying and storing offsets externally for exactly-once delivery. After tasks are initialized, they are started with a `Properties` object that contains the configuration the `Connector` created for the task. Once tasks are started, source tasks poll an external system and return lists of records that the worker sends to Kafka brokers. Sink tasks receive records from Kafka through the worker and are responsible for writing the records to an external system.

Workers

Kafka Connect's worker processes are the "container" processes that execute the connectors and tasks. They are responsible for handling the HTTP requests that define connectors and their configuration, as well as for storing the connector configuration in an internal Kafka topic, starting the connectors and their tasks, and passing the appropriate configurations along. If a worker process is stopped or crashes, other workers in a Connect cluster will recognize that (using the heartbeats in Kafka's consumer protocol) and reassign the connectors and tasks that ran on that worker to the remaining workers. If a new worker joins a Connect cluster, other workers will notice that and assign connectors or tasks to it to make sure load is balanced among all workers fairly. Workers are also responsible for automatically committing offsets for both source and sink connectors into internal Kafka topics and for handling retries when tasks throw errors.

The best way to understand workers is to realize that connectors and tasks are responsible for the "moving data" part of data integration, while the workers are responsible for the REST API, configuration management, reliability, high availability, scaling, and load balancing.

This separation of concerns is the main benefit of using the Connect API versus the classic consumer/producer APIs. Experienced developers know that writing code that reads data from Kafka and inserts it into a database takes maybe a day or two, but if you need to handle configuration, errors, REST APIs, monitoring, deployment, scaling up and down, and handling failures, it can take a few months to get everything right. And most data integration pipelines involve more than just the one source or target. So now consider that effort spent on bespoke code for just a database integration, repeated many times for other technologies. If you implement data copying with a connector, your connector plugs into workers that handle a bunch of complicated operational issues that you don't need to worry about.

Converters and Connect's data model

The last piece of the Connect API puzzle is the connector data model and the converters. Kafka's Connect API includes a data API, which includes both data objects and a schema that describes that data. For example, the JDBC source reads a column from a database and constructs a `Connect Schema` object based on the data types of the columns returned by the database. It then uses the schema to construct a `Struct` that contains all the fields in the database record. For each column, we store the column name and the value in that column. Every source connector does something similar—read an event from the source system and generate a

`Schema` and `Value` pair. Sink connectors do the opposite—get a `Schema` and `Value` pair and use the `Schema` to parse the values and insert them into the target system.

Though source connectors know how to generate objects based on the Data API, there is still a question of how `Connect` workers store these objects in Kafka. This is where the converters come in. When users configure the worker (or the connector), they choose which converter they want to use to store data in Kafka. At the moment, the available choices are primitive types, byte arrays, strings, Avro, JSON, JSON schemas, or Protobufs. The JSON converter can be configured to either include a schema in the result record or not include one—so we can support both structured and semistructured data. When the connector returns a Data API record to the worker, the worker then uses the configured converter to convert the record to an Avro object, a JSON object, or a string, and the result is then stored into Kafka.

The opposite process happens for sink connectors. When the Connect worker reads a record from Kafka, it uses the configured converter to convert the record from the format in Kafka (i.e., primitive types, byte arrays, strings, Avro, JSON, JSON schema, or Protobufs) to the Connect Data API record and then passes it to the sink connector, which inserts it into the destination system.

This allows the Connect API to support different types of data stored in Kafka, independent of the connector implementation (i.e., any connector can be used with any record type, as long as a converter is available).

Offset management

Offset management is one of the convenient services the workers perform for the connectors (in addition to deployment and configuration management via the REST API). The idea is that connectors need to know which data they have already processed, and they can use APIs provided by Kafka to maintain information on which events were already processed.

For source connectors, this means that the records the connector returns to the Connect workers include a logical partition and a logical offset. Those are not Kafka partitions and Kafka offsets but rather partitions and offsets as needed in the source system. For example, in the file source, a partition can be a file and an offset can be a line number or character number in the file. In a JDBC source, a partition can be a database table and the offset can be an ID or timestamp of a record in the table. One of the most important design decisions involved in writing a source connector is deciding on a good way to partition the data in the source system and to track offsets—this will impact the level of parallelism the connector can achieve and whether it can deliver at-least-once or exactly-once semantics.

When the source connector returns a list of records, which includes the source partition and offset for each record, the worker sends the records to Kafka brokers. If the brokers successfully acknowledge the records, the worker then stores the offsets of the records it sent to Kafka. This allows connectors to start processing events from the most recently stored offset after a restart or a crash. The storage mechanism is pluggable and is usually a Kafka topic; you can control the topic name with the `offset.storage.topic` configuration. In addition, Connect uses Kafka topics to store the configuration of all the connectors we've created and

the status of each connector—these use names configured by `config.storage.topic` and `status.storage.topic`, respectively.

Sink connectors have an opposite but similar workflow: they read Kafka records, which already have a topic, partition, and offset identifiers. Then they call the `connector put()` method that should store those records in the destination system. If the connector reports success, they commit the offsets they've given to the connector back to Kafka, using the usual consumer commit methods.

Offset tracking provided by the framework itself should make it easier for developers to write connectors and guarantee some level of consistent behavior when using different connectors.

Alternatives to Kafka Connect

So far we've looked at Kafka's Connect API in great detail. While we love the convenience and reliability the Connect API provides, it is not the only method for getting data in and out of Kafka. Let's look at other alternatives and when they are commonly used.

Ingest Frameworks for Other Datastores

While we like to think that Kafka is the center of the universe, some people disagree. Some people build most of their data architectures around systems like Hadoop or Elasticsearch. Those systems have their own data ingestion tools—Flume for Hadoop, and Logstash or Fluentd for Elasticsearch. We recommend Kafka's Connect API when Kafka is an integral part of the architecture and when the goal is to connect large numbers of sources and sinks. If you are actually building a Hadoop-centric or Elastic-centric system and Kafka is just one of many inputs into that system, then using Flume or Logstash makes sense.

GUI-Based ETL Tools

Old-school systems like Informatica, open source alternatives like Talend and Pentaho, and even newer alternatives such as Apache NiFi and StreamSets, support Apache Kafka as both a data source and a destination. If you are already using these systems—if you already do everything using Pentaho, for example—you may not be interested in adding another data integration system just for Kafka. They also make sense if you are using a GUI-based approach to building ETL pipelines. The main drawback of these systems is that they are usually built for involved workflows and will be a somewhat heavy and involved solution if all you want to do is get data in and out of Kafka. We believe that data integration should focus on faithful delivery of messages under all conditions, while most ETL tools add unnecessary complexity.

We do encourage you to look at Kafka as a platform that can handle data integration (with Connect), application integration (with producers and consumers), and stream processing. Kafka could be a viable replacement for an ETL tool that only integrates data stores.

Stream Processing Frameworks

Almost all stream processing frameworks include the ability to read events from Kafka and write them to a few other systems. If your destination system is supported and you already intend to use that stream pro-

cessing framework to process events from Kafka, it seems reasonable to use the same framework for data integration as well. This often saves a step in the stream processing workflow (no need to store processed events in Kafka—just read them out and write them to another system), with the drawback that it can be more difficult to troubleshoot things like lost and corrupted messages.

Summary

In this chapter we discussed the use of Kafka for data integration. Starting with reasons to use Kafka for data integration, we covered general considerations for data integration solutions. We showed why we think Kafka and its Connect API are a good fit. We then gave several examples of how to use Kafka Connect in different scenarios, spent some time looking at how Connect works, and then discussed a few alternatives to Kafka Connect.

Whatever data integration solution you eventually land on, the most important feature will always be its ability to deliver all messages under all failure conditions. We believe that Kafka Connect is extremely reliable—based on its integration with Kafka's tried-and-true reliability features—but it is important that you test the system of your choice, just like we do. Make sure your data integration system of choice can survive stopped processes, crashed machines, network delays, and high loads without missing a message. After all, at their heart, data integration systems only have one job—delivering those messages.

Of course, while reliability is usually the most important requirement when integrating data systems, it is only one requirement. When choosing a data system, it is important to first review your requirements (refer to "Considerations When Building Data Pipelines" for examples) and then make sure your system of choice satisfies them. But this isn't enough—you must also learn your data integration solution well enough to be certain that you are using it in a way that supports your requirements. It isn't enough that Kafka supports at-least-once semantics; you must be sure you aren't accidentally configuring it in a way that may end up with less than complete reliability.